E N G I N E E R I N G I N V E S T I G A T I O N

Research-to-Visualization Investigation Report

Decision-tree generation: structural validity, context relevance and provenance trust

Component: ShadowTalk AI — Research-to-Visualization pipeline (Batch 1–3)

Trigger: USDT exposure decision tree test screenshot (viz-8)

Method: Static analysis + executed 17-assertion harness (cases A–E)

Constraints: Investigation only — no production code modified

Date: November 13, 2026

Table of Contents

1. Executive Summary.....

2. Scope, Method and Evidence Base.....2

2.1 Scope.....2

2.2 Method and evidence.....2

2.3 Limitations.....2

3. Feature and Pipeline Architecture.....3

4. Screenshot Evidence and Interaction Sequence.....3

4.1 What the screenshot shows.....3

4.2 Establishing the actual interaction sequence.....3

5. Test Case Results (A–E).....4

6. Confirmed Defects and Root Causes.....4

CD-1 — Decision-branch contract is prompt-only (High).....4

CD-2 — Disconnected nodes are silently permitted and drawn as pseudo-start-nodes (High).....5

CD-3 — Topic-irrelevant context is classified “sufficient” (High).....5

CD-4 — Unconditional newest-message fallback injects unrelated content (High).....5

CD-5 — Cross-request provenance carry-over (High).....5

CD-6 — No allowlist on model-attached source URLs (Medium).....6

CD-7 — Untyped nodes mean different things to the validator and the renderer (Medium).....6

CD-8 — Duplicate edges, self-loops and cycles are structurally legal (Low).....6

7. Suspected Defects and Open Risks.....6

SD-1 — Edge-label ambiguity and overlap (Medium, partially confirmed).....7

SD-2 — Outcome semantics are unchecked (Medium).....7

SD-3 — First-node force-typing can mislabel the entry (Medium).....7

SD-4 — “1 source” on the original card is unverifiable from the screenshot (Low).....7

8. What Is Verified Working.....8

9. Reproduction Steps.....9
9.1 Harness reproduction (code level).....9
9.2 UI reproduction (application level).....9
10. Recommended Fixes (Not Implemented).....10
11. Conclusion.....10
Appendix A: Exact Files and Functions Responsible.....10
Appendix B: Investigation Harness Composition.....11

Note: This Table of Contents is generated via field codes. To ensure page number accuracy after editing, please right-click the TOC and select "Update Field."

1. Executive Summary

This report documents an investigation of the Research-to-Visualization feature of ShadowTalk AI, triggered by a test screenshot of a generated *USDT exposure decision tree* (docs/architecture/research-visualization/screenshots/viz-8-concept-decision-tree.png). The investigation covered the complete pipeline — intent detection, context resolution, gap assessment, model-assisted generation, schema validation, provenance assignment and SVG rendering — and executed the five requested test cases (A–E) as a 17-assertion harness against the real pipeline modules. No production code was modified, no schema was changed, and nothing was committed.

Overall verdict: the feature is structurally safe but not yet semantically trustworthy. The closed-schema security model is genuinely solid — the model can only author JSON, every candidate passes a zod validation gate, renderers treat all labels as plain text, and conceptual diagrams can never present a fabricated number as data. However, the decision-tree graph contract is enforced only by prompt wording and not by code, the context resolver injects conversation content that can be irrelevant to the current request, and model-attached source URLs are accepted without an allowlist check. All three areas produced reproducible defects, each confirmed by executed code paths rather than by inspecting rendered output.

The four findings with the highest impact are: **(1)** decision nodes may ship with fewer than two outgoing edges and without condition labels, which explains the single-branch “Reserves verified?” node in the screenshot; **(2)** disconnected nodes are silently permitted and are rendered in the root row as pseudo-start-nodes, which explains the floating “Review reserve reports” node; **(3)** any structured dataset captured anywhere in the session marks every later request as “sufficient” regardless of topic, so a USDT research block is fed as grounding context to an unrelated “stablecoin data platform” request, together with its source URL — reproducing exactly the “AI-EXTRACTED · 1 source” card combination shown in the screenshot; **(4)** when no context matches at all, the resolver unconditionally injects the newest assistant message as grounding evidence, even with zero keyword overlap.

On the screenshot’s timestamp question: the 12:40 PM prompt “Draw the architecture of a stablecoin data platform.” appears below the 12:35 PM USDT response because chat messages render in array order with per-message creation timestamps. The later prompt postdates the response and therefore **cannot have contaminated the USDT tree**; the contamination risk runs forward, from the earlier USDT research into the later architecture request. Recommended fixes are listed in Section 10 with priorities; none have been implemented.

2. Scope, Method and Evidence Base

2.1 Scope

The investigation targeted the conceptual (Batch 3) diagram pipeline with emphasis on decision trees: generation entry points in the chat page, the context resolver and gap assessor, the model bridge and prompt contracts, the visualization specification schema and its cross-field validation, the provenance policy, and the conceptual diagram renderer. Data-chart kinds (line, bar, pie, candlestick and related), storage mechanics and provider integrations were reviewed only where they interact with the decision-tree path.

2.2 Method and evidence

Three evidence channels were combined. First, static analysis of 21 source files with exact line references (Appendix A). Second, an executed test harness — a temporary vitest file, deleted after the run — that exercised the real modules (contextResolver, gapAnalysis, modelBridge, conceptualPipeline, spec validator and the React renderer) with a mocked model and research bridge; its 17 assertions are listed with results in Section 5 and Appendix B. Third, review of the shipped screenshot and of the feature's own architecture documents (PHASE1_AUDIT.md, BATCH2_REPORT.md, BATCH3_REPORT.md) to separate newly found defects from documented design intent.

Constraint compliance: investigation only. Production code, schemas and the visualization feature were left untouched; the working tree was verified clean afterward (git status: 0 changes); no commits or pushes were made. Semantic correctness is never claimed from rendering alone — every semantic claim in this report is backed by an executed code path or an exact code citation.

2.3 Limitations

The harness mocks the model and research calls, so findings about model behaviour are findings about what the pipeline permits a model to do, not observations about one specific provider. The screenshot's underlying session data was not available, so the exact node IDs and edge list of the original USDT tree are reconstructed from the visible labels, edge labels and layout rather than read from a persisted specification.

3. Feature and Pipeline Architecture

A visualization request travels through a deterministic routing layer into an AI-assisted conceptual pipeline. The stages below are the ones relevant to decision trees; file and function references are exact (line numbers as of commit 77368ac0 on feature/appearance-studio).

Table 1: Decision-tree generation pipeline, stage by stage

Stage	Location	Role
1. Intent detection	src/lib/visualization/intentDetector.ts — detectVisualizationIntent (L128–199)	Classifies create / refine / explain; “Draw the architecture of …” matches create pattern 5 (L85) without a type hint
2. Session wiring	src/pages/ChatbotPage.tsx — runVisualizationRequest (L1531–1543), toPipelineMessages (L1480–1491)	Passes the FULL session (welcome message filtered only) plus BYOK model and research calls
3. Routing	src/lib/visualization/chatPipeline.ts — handleVisualizationRequest (L525–541)	Decision-tree hints and conceptual ops route to handleConceptualRequest
4. Context resolution	src/lib/visualization/ contextResolver.ts — resolveResearchContext (L121–243)	Scores ALL prior AI messages by priority class, keyword overlap and recency; unconditional newest-message fallback (L182–189)
5. Gap assessment	src/lib/visualization/conceptual/ gapAnalysis.ts — assessGaps (L89–138), buildContextDigest (L38–59)	Verdict sufficient / needs-research / cannot-derive; digest = dataset lines + evidence excerpts + declared gaps
6. Deterministic build	gapAnalysis.ts — buildConceptFromDatasets (L165–327)	Roadmap / mindmap / architecture from structured datasets; no decision-tree builder exists — trees are always model-authored
7. Model generation	conceptual/conceptualPipeline.ts — generateConceptSpec (L176–245); conceptual/prompts.ts (L16–72)	Closed-schema JSON prompt; sources offered as “CITABLE SOURCES you may attach” (L64–67); one repair round on validation failure
8. Normalize + gate	conceptual/modelBridge.ts — normalizeConceptCandidate (L103–308), validateConceptCandidate (L323–337); spec.ts — validateVisualizationSpec (L788–873)	Kind coercion, string capping, unknown-key stripping, decision-tree cross-checks (L530–540)
9. Provenance finalize	conceptualPipeline.ts — finalizeConceptProvenance (L94–113), conceptOutcome (L115–137)	sources > 0 → ai-extracted; none → unverified + gap note; response text and source chips derive from the spec
10. Rendering	src/components/visualization/ ConceptualDiagramRenderer.tsx — layoutDecisionTree (L266–326), DecisionTreeSvg (L769–827); VisualizationCard.tsx (L51–55)	Re-parses the spec (parseVisualizationSpec), BFS level layout, SVG paths, kind-coloured nodes

Two properties of this architecture matter for the findings. First, the pipeline is **session-scoped, not topic-scoped**: stage 2 passes the entire session and stage 4 scores every prior

assistant message. Second, the **validation gate is the only semantic boundary**: everything the prompt promises ("every node kind decision must have ≥2 outgoing edges with condition labels", "labels must come from the supplied context", "sources: ONLY urls that were supplied to you") is enforced nowhere except by asking the model to comply.

4. Screenshot Evidence and Interaction Sequence

4.1 What the screenshot shows

The card is titled "USDT exposure decision tree" with an AI-EXTRACTED badge and a "1 source" chip; the response text above it reads "Provenance: synthesized by the model from the attached sources (labelled ai-extracted)." Five nodes are visible: "Peg under $0.99?" (amber, decision), "Review reserve reports" (grey, no edges attached), "Hold allocation" and "Reduce exposure" (green, outcomes), and "Reserves verified?" (amber, decision). Three edges are traceable: Peg—no→Hold, Peg—yes→Reserves verified?, and Reserves verified?—no→Reduce exposure. "Reserves verified?" therefore shows only one outgoing branch and no visible yes-outcome; "Review reserve reports" is completely disconnected; and the "yes" label of the long Peg→Reserves edge renders at the curve midpoint directly beside "Hold allocation", where it is easy to misread as an edge leaving "Hold allocation". The zoom indicator reads 87%.

4.2 Establishing the actual interaction sequence

The displayed prompt "Draw the architecture of a stablecoin data platform." carries a 12:40 PM timestamp, five minutes after the 12:35 PM response. Message ordering in the chat is array order, and each message is stamped with a creation Date at push time (ChatbotPage.tsx L1537 creates the placeholder AI message with new Date(); MessageBubble.tsx L522–524 renders format(message.timestamp, "MMM d, h:mm a")). The later-timestamped prompt is therefore simply the next user turn, sent after the USDT tree was answered. **It cannot have contaminated the 12:35 response.**

The contamination question must instead be asked forward: what session context fed the 12:40 architecture request, whose answer is not visible in the screenshot? Section 5 (case B) shows that the pipeline would resolve the earlier USDT research as primary evidence, classify sufficiency without checking topic overlap, and offer the USDT source URL to the model as an attachable citation. In other words, the screenshot's "AI-EXTRACTED · 1 source" combination is legitimate for the tree itself (assuming one source genuinely backed the USDT discussion), while the same mechanism makes the *next* response the contamination risk. Whether the original "1 source" was real cannot be verified from the screenshot alone; no persisted session was available for inspection (see Section 7, SD-4).

5. Test Case Results (A–E)

All five requested cases were executed as a 17-assertion vitest harness importing the real modules (resolveResearchContext, assessGaps, buildContextDigest, handleConceptualRequest with a mocked ConceptModelCall, validateVisualizationSpec and the rendered ConceptualDiagramRenderer). Every assertion below passed, which means every behaviour described as a defect was reproduced against the production code. The harness file was deleted after execution; reproduction steps are in Section 9.

Table 2: Executed results for test cases A–E (17 assertions, all passed)

ID	Assertion	Result / significance
A1	Well-formed tree (2 decisions, 4 labelled edges, 2 outcomes) passes validation	Baseline works as intended
A2	Decision node with a single outgoing edge passes	DEFECT — branch-factor rule is prompt-only (CD-1)
A3	Decision node with zero outgoing edges passes	DEFECT — dead-end decision allowed (CD-1)
A4	Decision edges without condition labels pass	DEFECT — labels optional in schema (CD-1)
A5	Fully untyped node list passes and its first node is force-typed “decision”	DEFECT — validator/renderer semantics diverge (CD-7)
A6	Duplicate edges and a self-loop edge pass	DEFECT — no uniqueness or self-loop check (CD-8)
A7	A cyclic “tree” passes	DEFECT — no acyclicity check (CD-8)
B1	Architecture request resolves the USDT message as primary evidence	CONFIRMED — session-scoped context (CD-3)
B2	assessGaps returns “sufficient” with topicOverlap=false (1/5 tokens)	DEFECT — datasets>0 short-circuits relevance (CD-3)
B3	Digest handed to the model contains USDT dataset + excerpt	CONFIRMED — irrelevant grounding delivered (CD-3)
B4	End-to-end: new architecture spec ships with the old USDT source, provenance ai-extracted, response says “1 source”	DEFECT — reproduces the screenshot card combination (CD-5)
B5	Model attaches a never-supplied URL; gate accepts it	DEFECT — no source allowlist (CD-6)
C1	“Decision tree from the previous research” resolves research-data evidence and a grounded digest	WORKS AS DESIGNED
D1	Unrelated conversation: newest AI answer injected with zero keyword overlap	DEFECT — unconditional fallback (CD-4)

ID	Assertion	Result / significance
D2	Truly empty session produces no evidence and provenance unverified	Correct boundary behaviour
E1	Intentionally disconnected node: validation ok=true, errors=[]	DEFECT — silently permitted, no recorded reason (CD-2)
E2	Rendered DOM: disconnected node's rect y equals the start decision's y	DEFECT — drawn in the root row, indistinguishable (CD-2)

6. Confirmed Defects and Root Causes

Each defect below was reproduced by an executed assertion (Section 5). Severity reflects impact on trustworthiness: High means the feature can present a misleading structure or attribution as validated fact.

CD-1 — Decision-branch contract is prompt-only (High)

The system prompt requires "every node kind decision must have ≥2 outgoing edges with condition labels" (conceptual/prompts.ts L43), but neither the schema nor the cross-field validator enforces it. decisionEdgeSchema declares the condition label optional (spec.ts L310–315), and validateDecisionKind (spec.ts L530–540) checks only three things: unique node ids, edge endpoints that reference existing nodes, and the presence of at least one node of kind "decision". Outgoing-edge counts per decision node, label presence, duplicate edges, self-loops and acyclicity are all unchecked; normalizeConceptCandidate likewise filters edges only for dangling references (modelBridge.ts L199–207). **Consequence:** a decision node with one branch and no labels validates cleanly — precisely the "Reserves verified?" shape in the screenshot, where the yes-branch outcome is missing and the tree answers a decision without presenting its alternative.

CD-2 — Disconnected nodes are silently permitted and drawn as pseudo-start-nodes (High)

validateDecisionKind has no reachability or connectivity check, so an isolated node produces ok=true with an empty error list (assertion E1) — permitted, but not intentional: neither spec.ts nor the Batch 3 report documents or records any reason for isolation. The renderer then places every unreachable node at level 0: layoutDecisionTree seeds its BFS from the first decision node (ConceptualDiagramRenderer.tsx L280) and assigns unmatched nodes `levelOf.get(n.id) ?? 0` (L294–297), which drops them into the top row beside the real start node, styled identically to any connected option node (L798–823). **Consequence:** "Review reserve reports" in the screenshot renders as a floating sibling of the root question

with no visual distinction, no caption and no warning — the viewer cannot tell an intentional side-note from a broken edge.

CD-3 — Topic-irrelevant context is classified “sufficient” (High)

assessGaps decides with `if (hasContext && (topicOverlap || context.datasets.length > 0))` (gapAnalysis.ts L103). The documented design is “sufficient: context has datasets/evidence AND topic overlap” (L84–85), but the implemented OR makes the topic check dead whenever one structured dataset exists anywhere in the session. Assertion B2 shows the request “Draw the architecture of a stablecoin data platform.” scoring 1/5 topic tokens against USDT research and still receiving verdict “sufficient”; B3 shows the digest actually handed to the model containing the USDT dataset line and the USDT excerpt. **Consequence:** every later diagram in a session inherits the first research topic as its grounding context, and the honesty rule “labels MUST come from the supplied research context” (prompts.ts L19) then actively pushes USDT vocabulary into unrelated diagrams.

CD-4 — Unconditional newest-message fallback injects unrelated content (High)

When no candidate scores, resolveResearchContext pushes the newest non-empty assistant message as text-match evidence with score 0 (contextResolver.ts L182–189) — regardless of keyword overlap. Assertion D1: in a session about Dante’s Divine Comedy, the request “Draw a decision tree for launching a product.” resolves primaryMessageId = the Dante answer, and the digest (via buildContextDigest) delivers its text as “RESEARCH CONTEXT available to ground labels”. Assertion D2 shows the only safe boundary: a completely empty session injects nothing. **Consequence:** test case D’s requirement — that a new unrelated conversation must not receive previous research — holds only for pristine sessions; any prior exchange, however unrelated, is injected.

CD-5 — Cross-request provenance carry-over (High)

conceptualCreateFlow builds the prompt’s attachable source list from context.citations (conceptualPipeline.ts L519–521) — the union of prior citations and any fresh research — and the model’s attached sources become the spec’s sources (modelBridge.ts L291–300), which drive both the AI-EXTRACTED badge (finalizeConceptProvenance, L94–113: any source → ai-extracted) and the “N source(s)” chip and source list (VisualizationContainer.tsx L312–335, SourceList). Assertion B4 runs the full flow end-to-end: a mocked model attaching the USDT URL to the new stablecoin-architecture spec produces provenance ai-extracted, sources.length = 1 pointing at the old URL, and the response text “1 source attached”. **Consequence:** the exact card combination in the screenshot can be produced for a request that has nothing to do with the attached source. The

label “ai-extracted” is honest about synthesis, but the source chip implies the source substantiates this diagram.

CD-6 — No allowlist on model-attached source URLs (Medium)

The prompt restricts attachments to “ONLY urls that were supplied to you” (prompts.ts L36, L64–67), but normalizeConceptCandidate accepts every http(s) URL the model emits (modelBridge.ts L291–300) with no cross-check against the supplied citations. Assertion B5: a fabricated URL (https://made-up-source.example.org/report.pdf) passes the gate and is rendered by SourceList as a clickable link with name and timestamps. **Consequence:** a hallucinated or adversarial completion can dress an unverified diagram in a plausible-looking citation; the http(s) scheme check blocks javascript:/data: but not fabrication.

CD-7 — Untyped nodes mean different things to the validator and the renderer (Medium)

decisionNodeSchema makes kind optional (spec.ts L304–308). validateDecisionKind counts untyped nodes as decisions for its “at least one decision” rule (`(n.kind ?? “decision”)`, spec.ts L537); the renderer instead types every untyped node as “option” (`node.kind ?? “option”`, renderer L306) and paints options grey; and when the model omits kinds entirely, normalizeConceptCandidate force-types the FIRST node as decision (modelBridge.ts L208–211) even if it is semantically an action item rather than a question. **Consequence:** the same spec can validate as containing a decision while visually presenting none; branch-factor enforcement keyed on kind would also be fooled.

CD-8 — Duplicate edges, self-loops and cycles are structurally legal (Low)

Assertions A6–A7: two edges with the same from/to (labelled differently) render as overlapping curves; a self-loop folds back on the same node; a 2-cycle renders as two stacked opposing curves. None are rejected, and the deterministic “simplify” BFS pruning (conceptualPipeline.ts L383–413) follows adjacency blindly, so cycles can also affect which nodes survive simplification. Impact is cosmetic-to-moderate, but it widens the gap between the name “decision-tree” and what the kind accepts.

7. Suspected Defects and Open Risks

SD-1 — Edge-label ambiguity and overlap (Medium, partially confirmed)

DecisionTreeSvg draws each condition label at the exact curve midpoint ((x1+x2)/2, midY−4) with no collision handling (ConceptualDiagramRenderer.tsx L785–789). The screenshot's confusing "yes" placement next to "Hold allocation" is consistent with this mechanism — the label belongs to the long Peg→Reserves edge — and assertion E2 confirms the layout geometry that produces such long diagonals. What is not fully confirmed is the worst-case behaviour (label overlapping a node rectangle) on dense trees; that requires rendering a larger corpus of generated trees, which was out of scope for a read-only investigation.

SD-2 — Outcome semantics are unchecked (Medium)

Nothing requires decision branches to terminate in outcome nodes, or leaf nodes to be outcomes; "outcome" is purely a colour choice in the renderer. A tree can legally end a branch on another decision (as "Reserves verified?" effectively does with its missing branch) or on an option node. The prompt's node vocabulary implies the invariant; the schema does not state it.

SD-3 — First-node force-typing can mislabel the entry (Medium)

When the model omits kinds, modelBridge.ts L208–211 promotes nodes[0] to "decision". Both the BFS layout (renderer L280) and any future branch-factor validation would then treat that node as the entry question even when the label reads like an action ("Review reserve reports"). Not observed in the screenshot (its first node is a genuine question), hence classified suspected; the mechanism is verified by code reading and assertion A5.

SD-4 — "1 source" on the original card is unverifiable from the screenshot (Low)

The displayed source count is faithful to spec.sources.length by construction (VisualizationContainer.tsx L335), but whether the original USDT tree's single source was a genuinely captured research citation or a model-attached URL cannot be determined without the persisted session. CD-5/CD-6 show both paths exist. Verifying real deployments requires exporting a session's message list and inspecting the message-level citations alongside the spec.

8. What Is Verified Working

The trustworthiness verdict must be balanced against what the investigation confirmed to be solid. These properties were verified by code reading and, where noted, by executed assertions:

- Closed schema and no injection surface: every candidate is zod-parsed with unknown keys stripped recursively (spec.ts L882–888), strings are length-capped at normalization (modelBridge.ts L95, L103–308), and renderers emit labels only as SVG text children that React escapes; no code/HTML path exists from model output to the DOM.
- Honest failure modes: JSON extraction tolerates fences and prose (modelBridge.ts L58–72); one repair round feeds exact validation errors back (conceptualPipeline.ts L216–242); persistent failure produces an explicit refusal text instead of a diagram; no numeric data surface exists on conceptual kinds at all.
- Provenance ceiling: finalizeConceptProvenance can only raise a spec to ai-extracted, never to source-verified, and adds a synthesis note (L94–113); unverified requires an acknowledging gap note (spec.ts L868–870). The response wording in the screenshot matches conceptOutcome's template exactly (conceptualPipeline.ts L115–137).
- Renderer fidelity: the decision-tree renderer displays exactly the validated specification — nodes, kinds, edges, edge labels and their direction (source bottom-centre to target top-centre with an arrowhead) map 1:1 to the spec (assertions E2 and C1; layout and SVG sections L266–326, L769–827). Label truncation to 22–24 characters per line is a display choice, not content loss in the spec.
- Intended grounding path works: case C confirmed that "decision tree from the previous research" resolves research-data evidence with a grounded digest — the core feature goal is achieved when the request and the context genuinely match.
- Interaction timestamps are per-message creation times rendered in array order (ChatbotPage.tsx L1537; MessageBubble.tsx L522–524) — the sequence ambiguity in the screenshot is resolved by Section 4, not a defect.

9. Reproduction Steps

9.1 Harness reproduction (code level)

The harness was a temporary vitest file placed at src/lib/visualization/__tests__/ and removed after execution. To reproduce: create a scratch *.test.tsx importing resolveResearchContext (contextResolver.ts), assessGaps / buildContextDigest / formatDigestForPrompt (gapAnalysis.ts), handleConceptualRequest with a vi.fn() model returning a fixed JSON candidate (conceptualPipeline.ts + modelBridge.ts), validateVisualizationSpec (spec.ts) and render(ConceptualDiagramRenderer) from @testing-library/react. Fixtures: a USDT session message carrying researchData (dataset + one source) and citations; an unrelated literature session with plain text only. Then assert the

behaviours listed in Table 2 — each one is a single expect() against the named module. The renderer assertion compares rect y-attributes of the disconnected node and the start node.

9.2 UI reproduction (application level)

- Case A/B (contamination + provenance): open /chatbot, send “Research USDT exposure risk and reserves” and let a research tool return at least one cited source; then send “Draw the architecture of a stablecoin data platform.” with a BYOK key configured. The generated architecture card can attach the USDT source and shows the AI-EXTRACTED badge with “1 source”; Expand → Sources & timestamps lists the old URL (VisualizationContainer.tsx SourceList).
- Case C (intended reuse): in the same session ask for “a decision tree from the previous research” — the tree’s labels track the USDT research (expected behaviour).
- Case D (unrelated injection): start a fresh conversation on any non-research topic, then send “Draw a decision tree for launching a product.” With a BYOK key, expand the generation prompt (mock the model call or inspect request logs): the user prompt’s RESEARCH CONTEXT section contains the unrelated answer text.
- Case E (disconnected node): in an expanded diagram ask “add a note node Review reserve reports” — or craft the spec through the harness — and observe the node appear in the top row with normal styling, no warning, and validation passing (Expand dialog shows no defects).

10. Recommended Fixes (Not Implemented)

Fixes are listed with priority, target location and expected effect. None have been applied; production code and schemas are unchanged.

Table 3: Prioritized recommendations

Prio	Fix	Target	Expected effect
P0-1	Enforce the graph contract in validateDecisionKind: every decision node ≥2 outgoing edges, each with a non-empty label; reject duplicate edges and self-loops; require acyclicity and reachability from the entry decision; for an intentionally isolated node require an explicit acknowledged reason in gaps (mirroring the unverified-gap pattern of spec.ts L868–870)	spec.ts validateDecisionKind (L530–540); optionally decisionEdgeSchema	Single-branch and floating nodes are rejected or explicitly acknowledged; the gate matches the prompt

Prio	Fix	Target	Expected effect
P0-2	Repair the sufficiency check: replace `topicOverlap || context.datasets.length > 0` with a requirement of genuine topic overlap, or an explicit referential request (“this / the previous research”); attach the computed overlap and the chosen evidence ids to the digest	gapAnalysis.ts assessGaps (L89–138), buildContextDigest (L38–59)	Unrelated requests trigger scoped research instead of inheriting the session’s first topic
P0-3	Make the zero-candidate fallback conditional: inject the newest AI message only when the request is referential (no content keywords) or the message carries research-data/citations; never inject plain text with zero keyword overlap	contextResolver.ts (L182–189)	Test case D passes: unrelated conversations receive no injected grounding
P0-4	Source allowlist: filter model-attached sources to the URLs supplied in the prompt (mergedCitations); drop or downgrade others with a gap note; drive the source-count chip from verified sources only	modelBridge.ts (L291–300); conceptualPipeline.ts (L519–521); VisualizationContainer.tsx source chip	“1 source” can no longer mean a fabricated or carried-over URL
P1-1	Require explicit kinds on all decision-tree nodes at normalization (reject or sensibly infer with a recorded note); align the validator’s untyped-as-decision rule with the renderer’s untyped-as-option rendering	modelBridge.ts (L179–214); spec.ts (L537); renderer (L306)	Validator semantics, renderer styling and branch checks agree
P1-2	Outcome semantics: validate that every decision branch terminates in a leaf and prefer leaf = outcome; renderer marks unreachable nodes distinctly (dashed border, separate “unlinked” row with caption) and offsets edge labels to avoid node/edge overlap	spec.ts; ConceptualDiagramRenderer.tsx (L280–297, L785–789)	Structural completeness becomes visible; the screenshot’s two ambiguities disappear
P1-3	Surface context provenance on the card: record which message ids fed the digest (verdict, topicOverlap, evidence ids) in the spec or the outcome notes so “grounded in what?” is inspectable per card	conceptualPipeline.ts (finalizeConceptProvenance / conceptOutcome)	Trust review per card becomes possible without session exports

Prio	Fix	Target	Expected effect
P2-1	Renderer hardening: deterministic BFS child ordering (sort by id or spec order), label collision pass, entry-node "START" cue	ConceptualDiagramRenderer.tsx	Stable, readable layouts across regenerations
P2-2	Promote the 17 harness assertions to permanent regression tests (context relevance, gate strictness, provenance allowlist, disconnected-node rendering)	src/lib/visualization/__tests__/	Fixes cannot silently regress

Sequencing note: P0-1 and P0-2/P0-3 are independent and can land in either order; P0-4 depends only on threading the supplied-URL list into normalizeConceptCandidate's options. P1-1 should precede any stricter kind-keyed validation, otherwise untyped nodes bypass it (CD-7).

11. Conclusion

Against the investigation's three-part goal: **visually impressive** — yes, and the renderer is a faithful, escape-only projector of the validated specification; **structurally valid** — at the security boundary yes (closed schema, no injection, honest refusals), but at the graph-semantics level no: the decision-tree contract exists only in the prompt, so disconnected nodes, single-branch decisions, unlabeled conditions, duplicates and cycles all validate (CD-1, CD-2, CD-7, CD-8); **contextually relevant and trustworthy** — partially: intended reuse (case C) works, but relevance is not enforced (CD-3, CD-4) and provenance chips can reference sources that are old, carried-over or entirely fabricated (CD-5, CD-6).

The screenshot's four visible problems are therefore explained by confirmed root causes rather than rendering errors: the disconnected node is a validation gap made visible by the level-0 placement rule; the confusing condition labels are an edge-midpoint drawing choice over long BFS diagonals; the unclear outcomes reflect missing branch-completeness enforcement; and the AI-EXTRACTED / 1-source combination is the provenance system working as coded, with its weakest link (source allowlisting) exposed. The feature earns its safety story but not yet its trust story; the P0 set in Section 10 closes the gap with changes confined to two validators, the resolver/assessor and the source-normalization pass.

Appendix A: Exact Files and Functions Responsible

Table 4: Responsibility map (paths relative to repository root; lines at commit 77368ac0)

Defect	File	Function / location
CD-1	src/lib/visualization/spec.ts	decisionEdgeSchema (L310–315, label optional); validateDecisionKind (L530–540, no branch/label/connectivity checks)
CD-1	src/lib/visualization/conceptual/prompts.ts	conceptCreateSystemPrompt L43 (the ≥2-edges rule exists only here)
CD-2	src/lib/visualization/spec.ts	validateDecisionKind (no reachability check)
CD-2	src/components/visualization/ConceptualDiagramRenderer.tsx	layoutDecisionTree L280 (BFS seed), L292–297 (`?? 0` level for unreachable nodes); DecisionTreeSvg L798–823 (uniform styling)
CD-3	src/lib/visualization/conceptual/gapAnalysis.ts	assessGaps L103 (`topicOverlap || datasets.length > 0`); buildContextDigest L38–59
CD-4	src/lib/visualization/contextResolver.ts	resolveResearchContext L182–189 (unconditional newest-AI-message fallback)
CD-5	src/lib/visualization/conceptual/conceptualPipeline.ts	conceptualCreateFlow L519–521 (merged citations → prompt sources); finalizeConceptProvenance L94–113; conceptOutcome L115–137
CD-5	src/components/visualization/VisualizationContainer.tsx	source chip + Sources & timestamps (L312–335), SourceList
CD-6	src/lib/visualization/conceptual/modelBridge.ts	normalizeConceptCandidate L291–300 (http(s) filter only, no allowlist)
CD-7	src/lib/visualization/spec.ts + modelBridge.ts + renderer	spec.ts L537 (`?? “decision”`); modelBridge.ts L208–211 (force-type nodes[0]); renderer L306 (`?? “option”`)
CD-8	src/lib/visualization/spec.ts	validateDecisionKind (no duplicate/self-loop/cycle checks); conceptualPipeline.ts pruneConceptSpec L383–413 (BFS over adjacency)
Routing / context scope	src/pages/ChatbotPage.tsx	detectVisualizationIntent call L1697–1712; runVisualizationRequest L1531–1543; toPipelineMessages L1480–1491 (full session)
Timestamp mechanism	src/pages/ChatbotPage.tsx + src/components/chat/MessageBubble.tsx	L1537 (new Date() stamp); L522–524 (per-message time render)

Appendix B: Investigation Harness Composition

The scratch harness (deleted after execution) contained one describe block per test case and imported only production modules — no production file was edited, and no schema was

altered. Its 17 assertions are enumerated in Table 2; the fixture sessions were: (1) a USDT research exchange whose assistant message carries one researchData block (dataset “USDT reserve composition”, one source URL) plus a matching citation; (2) a literature exchange (“Who wrote the Divine Comedy?”) with plain assistant text and no research artifacts; (3) two architecture decision-tree candidates (one well-formed, one with an isolated node) and two model responses for the end-to-end flow (one attaching the old USDT URL, one attaching a fabricated URL). The renderer assertion rendered ConceptualDiagramRenderer directly and compared SVG rect y-attributes.

Compliance statement: during the investigation the repository working tree was verified clean (git status: no modifications); the only artifacts created were the temporary harness file (removed) and this report, which lives outside the repository. No commits, pushes, schema edits or feature changes were made, and no claim of semantic correctness is based on rendered appearance alone.